

Introdução à Programação na Web / Programação na Internet

Inverno de 2024/2025

* Teste Global de Época de Recurso (28 de Janeiro de 2025) - Duração 1h30

NÚMERO _____ NOME _____

GRUPO 1 [4 valores]

Nas seguintes questões, selecione a única opção verdadeira, escrevendo-a na folha de teste. Uma resposta correta conta a totalidade da cotação da pergunta; uma resposta incorreta desconta 25% da cotação da pergunta ao total do grupo; uma questão não respondida conta 0 valores.

- O método then() de Promise retorna?
 - undefined
 - null
 - Promise
 - String
- Ao processar um pedido para a criação de um recurso, na qual são enviados parâmetros inválidos, o status code adequado é:
 - 400
 - 200
 - 303
 - 500
- Ao analisar o URL <http://www.example.com/products?category=electronics>, qual das seguintes afirmações é verdadeira?
 - O path neste URL é /products?category=electronics
 - O path neste URL é <http://www.example.com>
 - A query string tem um parâmetro
 - O parâmetro products tem o valor category=electronics
- No protocolo HTTP, todas as mensagens com body têm o header:
 - Content-type
 - Body
 - Authorization
 - Location
- No protocolo HTTP, um pedido de método GET para o recurso <http://example.com/products/123/update> :
 - Solicita a atualização do recurso com URI <http://example.com/products/123>.
 - Solicita uma representação do recurso com URI <http://example.com/products/123>.
 - Solicita uma representação do recurso com URI <http://example.com/products/123/update>
 - Solicita a atualização do recurso com URI <http://example.com/products/123/update>
- Ao final de quanto tempo aparece a mensagem END?

- Imediatamente
- Nunca aparece
- Após 2 segundos
- Exatamente aos 2 segundos

```
let end = false;
function waitForEnd() {
  while (!end) {
    console.log(`waiting...`)
  }
}
console.log("BEGIN");
setTimeout(() => { end = true }, 2000);
waitForEnd();
console.log("END");
```

Introdução à Programação na Web / Programação na Internet

Inverno de 2024/2025

* Teste Global de Época de Recurso (28 de Janeiro de 2025) - Duração 1h30

NÚMERO _____ NOME _____

GRUPO 2 [4 valores]

Faça a correspondência entre o que está no lado esquerdo da tabela e o que lhe está relacionado no lado direito da tabela (identificado por letras). Escreva a letra correspondente na primeira coluna da tabela. Se por exemplo considerar que a primeira linha do lado esquerdo está relacionada com a opção C do lado direito, escreva C na primeira coluna desta linha.

1. Na linguagem CSS:

☐	a	☐	A	seleciona todos os elementos de um determinado tipo
☐	#funny	☐	B	seleciona todos os elementos com uma classe específica descendente de um elemento
☐	div .danger	☐	C	seleciona todos os elementos de uma determinada classe
☐	.danger	☐	D	seleciona o elemento com um id específico

2. Num array em Javascript:

☐	o método filter	☐	A	retorna undefined
☐	o método forEach	☐	B	retorna um novo array do mesmo tamanho do array original
☐	o método find	☐	C	retorna um elemento do array ou undefined
☐	o método map	☐	D	retorna um novo array com tamanho menor ou igual ao do array original

3. Em Javascript, uma função que:

☐	retorne uma promise	☐	A	tem de ser async
☐	use await	☐	B	pode não usar o await
☐	seja async	☐	C	retorna sempre uma promise
☐	não seja async	☐	D	nunca pode usar await

4. Foi realizado o pedido GET para o URL `http://somesite/a/1/c/2?a=5&d=6` numa aplicação express com a seguinte rota registada: `app.get("/a/:b/c/:d/", function (req, rsp) { . . . })`. Na implementação do handler:

☐	req.params.a	☐	A	undefined
☐	req.query.a	☐	B	Produz o valor 2
☐	req.params.d	☐	C	Produz o valor 5
☐	req.params.b	☐	D	Produz o valor 1

5. Na definição dos métodos HTTP, o pedido:

☐	Com método DELETE	☐	A	é idempotente e normalmente tem body
☐	Com método POST	☐	B	não é seguro e normalmente não tem body
☐	Com método GET	☐	C	não é idempotente
☐	Com método PUT	☐	D	é seguro

6. Na linguagem HTML

☐	Texto	☐	A	tem atributos
☐	Um elemento	☐	B	tem apenas um elemento raiz
☐	Um atributo	☐	C	nunca tem descendentes
☐	Um documento	☐	D	tem um valor

Introdução à Programação na Web / Programação na Internet

Inverno de 2024/2025

** Teste Global de Época de Recurso (28 de Janeiro de 2025) - Duração 1h30

NÚMERO _____ NOME _____

GRUPO 1 [4 valores]

Nas seguintes questões, selecione a única opção verdadeira, escrevendo-a na folha de teste. Uma resposta correta conta a totalidade da cotação da pergunta; uma resposta incorreta desconta 25% da cotação da pergunta ao total do grupo; uma questão não respondida conta 0 valores.

1. O método then() de Promise retorna?
 - A. undefined
 - B. null
 - C. Promise
 - D. String
2. Ao processar um pedido para a criação de um recurso, na qual são enviados parâmetros inválidos, o status code adequado é:
 - A. 400
 - B. 200
 - C. 303
 - D. 500
3. Ao analisar o URL <http://www.example.com/products?category=electronics>, qual das seguintes afirmações é verdadeira?
 - A. O path neste URL é /products?category=electronics
 - B. O path neste URL é <http://www.example.com>
 - C. A query string tem um parâmetro
 - D. O parâmetro products tem o valor category=electronics
4. No protocolo HTTP, todas as mensagens com body têm o header:
 - A. Content-type
 - B. Body
 - C. Authorization
 - D. Location
5. No protocolo HTTP, um pedido de método GET para o recurso <http://example.com/products/123/update> :
 - A. Solicita a atualização do recurso com URI <http://example.com/products/123>.
 - B. Solicita uma representação do recurso com URI <http://example.com/products/123>.
 - C. Solicita uma representação do recurso com URI <http://example.com/products/123/update>
 - D. Solicita a atualização do recurso com URI <http://example.com/products/123/update>
6. Ao final de quanto tempo aparece a mensagem END?
 - E. Imediatamente
 - F. Nunca aparece
 - G. Após 2 segundos
 - H. Exatamente aos 2 segundos

```
let end = false;
function waitForEnd() {
  while (!end) {
    console.log('waiting...')
  }
}
console.log("BEGIN");
setTimeout(() => { end = true }, 2000);
waitForEnd();
console.log("END");
```

NÚMERO _____ NOME _____

GRUPO 2 [4 valores]

Faça a correspondência entre o que está no lado esquerdo da tabela e o que lhe está relacionado no lado direito da tabela (identificado por letras). Escreva a letra correspondente na primeira coluna da tabela. Se por exemplo considerar que a primeira linha do lado esquerdo está relacionada com a opção C do lado direito, escreva C na primeira coluna desta linha.

1. Num array em Javascript:

☐ o método map	A	retorna um novo array com tamanho menor ou igual ao do array original
☐ o método forEach	B	retorna um elemento do array ou undefined
☐ o método filter	C	retorna um novo array do mesmo tamanho do array original
☐ o método find	D	retorna undefined

2. Em Javascript, uma função que:

☐ seja async	A	retorna sempre uma promise
☐ use await	B	nunca pode usar await
☐ retorne uma promise	C	tem de ser async
☐ não seja async	D	pode não usar o await

3. Foi realizado o pedido GET para o URL `http://somesite/a/1/c/2?a=5&d=6` numa aplicação express com a seguinte rota registada: `app.get("/a/:b/c/:d", function (req, rsp) { . . . })`. Na implementação do handler:

☐ req.params.a	A	Produz o valor 1
☐ req.params.b	B	Produz o valor 5
☐ req.params.d	C	Produz o valor 2
☐ req.query.a	D	undefined

4. Na definição dos métodos HTTP, o pedido:

☐ Com método POST	A	é idempotente e normalmente tem body
☐ Com método PUT	B	não é seguro e normalmente não tem body
☐ Com método GET	C	não é idempotente
☐ Com método DELETE	D	é seguro

5. Na linguagem HTML

☐ Um documento	A	nunca tem descendentes
☐ Um elemento	B	tem um valor
☐ Um atributo	C	tem atributos
☐ Texto	D	tem apenas um elemento raiz

6. Na linguagem CSS:

☐ .danger	A	seleciona todos os elementos de um determinado tipo
☐ #funny	B	seleciona todos os elementos de uma determinada classe
☐ div .danger	C	seleciona todos os elementos com uma classe específica descendente de um elemento
☐ a	D	seleciona o elemento com um id específico

NÚMERO _____ NOME _____

GRUPO 1 [4 valores]

Nas seguintes questões, selecione a única opção verdadeira, escrevendo-a na folha de teste. Uma resposta correta conta a totalidade da cotação da pergunta; uma resposta incorreta desconta 25% da cotação da pergunta ao total do grupo; uma questão não respondida conta 0 valores.

1. Ao processar um pedido para a criação de um recurso, na qual são enviados parâmetros inválidos, o status code adequado é:
A. 400
B. 200
C. 303
D. 500
 2. Ao analisar o URL <http://www.example.com/products?category=electronics>, qual das seguintes afirmações é verdadeira?
A. O path neste URL é </products?category=electronics>
B. O path neste URL é <http://www.example.com>
C. A query string tem um parâmetro
D. O parâmetro products tem o valor category=electronics
 3. No protocolo HTTP, todas as mensagens com body têm o header:
A. Content-type
B. Body
C. Authorization
D. Location
 4. Ao final de quanto tempo aparece a mensagem END?
A. Exatamente aos 2 segundos
B. Após 2 segundos
C. Nunca aparece
D. Imediatamente
- ```
let end = false;
function waitForEnd() {
 while (!end) {
 console.log('waiting...')
 }
}
console.log("BEGIN");
setTimeout(() => { end = true; }, 2000);
waitForEnd();
console.log("END");
```
5. No protocolo HTTP, um pedido de método GET para o recurso <http://example.com/products/123/update> :  
A. Solicita a atualização do recurso com URI <http://example.com/products/123>.  
B. Solicita uma representação do recurso com URI <http://example.com/products/123>.  
C. Solicita uma representação do recurso com URI <http://example.com/products/123/update>  
D. Solicita a atualização do recurso com URI <http://example.com/products/123/update>
  6. O método then() de Promise retorna?  
A. undefined  
B. null  
C. Promise  
D. String

**Introdução à Programação na Web / Programação na Internet**

Inverno de 2023/2024

\*\*\* Teste Global de Época Normal (10 de Janeiro de 2024) - Duração 1h30

NÚMERO \_\_\_\_\_ NOME \_\_\_\_\_

**GRUPO 2 [4 valores]**

Faça a correspondência entre o que está no lado esquerdo da tabela e o que lhe está relacionado no lado direito da tabela (identificado por letras). Escreva a letra correspondente na primeira coluna da tabela. Se por exemplo considerar que a primeira linha do lado esquerdo está relacionada com a opção C do lado direito, escreva C na primeira coluna desta linha.

1. Num array em Javascript:

|                                           |                            |                                                                       |
|-------------------------------------------|----------------------------|-----------------------------------------------------------------------|
| <input type="checkbox"/> o método map     | <input type="checkbox"/> A | retorna um novo array com tamanho menor ou igual ao do array original |
| <input type="checkbox"/> o método forEach | <input type="checkbox"/> B | retorna um elemento do array ou undefined                             |
| <input type="checkbox"/> o método filter  | <input type="checkbox"/> C | retorna um novo array do mesmo tamanho do array original              |
| <input type="checkbox"/> o método find    | <input type="checkbox"/> D | retorna undefined                                                     |

2. Em Javascript, uma função que:

|                                              |                            |                            |
|----------------------------------------------|----------------------------|----------------------------|
| <input type="checkbox"/> seja async          | <input type="checkbox"/> A | retorna sempre uma promise |
| <input type="checkbox"/> use await           | <input type="checkbox"/> B | nunca pode usar await      |
| <input type="checkbox"/> retorne uma promise | <input type="checkbox"/> C | tem de ser async           |
| <input type="checkbox"/> não seja async      | <input type="checkbox"/> D | pode não usar o await      |

3. Foi realizado o pedido GET para o URL `http://somesite/a/1/c/2?a=5&d=6` numa aplicação express com a seguinte rota registada: `app.get("/a/:b/c/:d", function (req, rsp) { . . . })`. Na implementação do handler:

|                                       |                            |                  |
|---------------------------------------|----------------------------|------------------|
| <input type="checkbox"/> req.params.a | <input type="checkbox"/> A | Produz o valor 1 |
| <input type="checkbox"/> req.params.b | <input type="checkbox"/> B | Produz o valor 5 |
| <input type="checkbox"/> req.params.d | <input type="checkbox"/> C | Produz o valor 2 |
| <input type="checkbox"/> req.query.a  | <input type="checkbox"/> D | undefined        |

4. Na definição dos métodos HTTP, o pedido:

|                                            |                            |                                         |
|--------------------------------------------|----------------------------|-----------------------------------------|
| <input type="checkbox"/> Com método POST   | <input type="checkbox"/> A | é idempotente e normalmente tem body    |
| <input type="checkbox"/> Com método PUT    | <input type="checkbox"/> B | não é seguro e normalmente não tem body |
| <input type="checkbox"/> Com método GET    | <input type="checkbox"/> C | não é idempotente                       |
| <input type="checkbox"/> Com método DELETE | <input type="checkbox"/> D | é seguro                                |

5. Na linguagem HTML

|                                       |                            |                             |
|---------------------------------------|----------------------------|-----------------------------|
| <input type="checkbox"/> Um documento | <input type="checkbox"/> A | nunca tem descendentes      |
| <input type="checkbox"/> Um elemento  | <input type="checkbox"/> B | tem um valor                |
| <input type="checkbox"/> Um atributo  | <input type="checkbox"/> C | tem atributos               |
| <input type="checkbox"/> Texto        | <input type="checkbox"/> D | tem apenas um elemento raiz |

6. Na linguagem CSS:

|                                      |                            |                                                                                   |
|--------------------------------------|----------------------------|-----------------------------------------------------------------------------------|
| <input type="checkbox"/> .danger     | <input type="checkbox"/> A | seleciona todos os elementos de um determinado tipo                               |
| <input type="checkbox"/> #funny      | <input type="checkbox"/> B | seleciona todos os elementos de uma determinada classe                            |
| <input type="checkbox"/> div .danger | <input type="checkbox"/> C | seleciona todos os elementos com uma classe específica descendente de um elemento |
| <input type="checkbox"/> a           | <input type="checkbox"/> D | seleciona o elemento com um id específico                                         |

**Introdução à Programação na Web / Programação na Internet**

Inverno de 2023/2024

\*\*\* Teste Global de Época Normal (10 de Janeiro de 2024) - Duração 1h30

---

NÚMERO \_\_\_\_\_ NOME \_\_\_\_\_

---



**GRUPO 3 [7 valores]**

1. [4] Considere o seguinte código em JavaScript.

```
function waitAndMultiply(value, multiplier, ms) {
 return new Promise((resolve) => {
 setTimeout(() => resolve(value * multiplier), ms);
 });
}

const startTime = Date.now();

function dif(time) {
 return Math.round((Date.now() - time) / 1000)
}

function executeProcess() {
 return waitAndMultiply(3, 2, 1000)
 .then((result1) => {
 console.log(`Step 1: res=${result1} (after ${dif(startTime)} seconds)`);
 return Promise.all([waitAndMultiply(result1, 4, 1500), waitAndMultiply(result1, 5, 5000)])
 })
 .then((result2) => {
 console.log(`Step 2: res=${result2[0]} and ${result2[1]} (after ${dif(startTime)} seconds)`);
 return result2[0] + result2[1]
 })
}

executeProcess().then(result3 => console.log(`Final Result ${result3}`))
console.log(`Execution started at ${new Date(startTime).toLocaleTimeString()}`);
```

- [1.5] Qual o *output* do programa?
  - [2.5] Crie uma versão alternativa de `executeProcess()` utilizando `async-await`, que apresente o mesmo *output* que a implementação anterior, mantendo a utilização do `Promise.all()`.
2. [3] Implemente a função `changeFetch` que intercepta as chamadas feitas à função `fetch` do JavaScript. O comportamento da função `changeFetch` deve ser o seguinte:
- A função `changeFetch` deve substituir a função `fetch` do Javascript por uma versão que conta quantas vezes cada método HTTP foi chamado (GET, POST, DELETE, PUT).
  - Cada vez que um pedido for feito, a função deve incrementar o contador correspondente ao método HTTP utilizado (GET, POST, DELETE ou PUT). O contador deve ser armazenado num objeto COUNTERS, onde as propriedades são os métodos HTTP e os valores são os contadores dos pedidos.
  - A nova função `fetch` (após ser substituída) deve continuar a fazer os pedidos, ou seja, ela deve chamar a implementação original do `fetch` após registar o pedido.

```
const COUNTERS = { GET : 0, POST : 0, PUT : 0, DELETE : 0 } //1 20%

function changeFetch() {
 const oldFetch = fetch //2 20%
 function myFetch(uri, options = {}){
 const method = options.method || 'GET'
 ++COUNTERS[method] //3 20%
 return oldFetch(uri, options) //4 20%
 }
 fetch = myFetch //5 20%
}

async function run() {
 const joke1 = await fetch('https://api.chucknorris.io/jokes/random')
 console.log(joke1)
 const joke2 = await fetch('https://api.chucknorris.io/jokes/random')
 console.log(joke2)
}

changeFetch()
run().then(()=>console.log(COUNTERS))
```



**GRUPO 4 [5 valores]**

3. Numa aplicação Node.js em Express, pretende-se criar um módulo logHttp que mostra na consola os detalhes do pedido e da respetiva resposta. Abaixo estão 2 pedidos HTTP e as respectivas mensagens na consola do servidor, bem como uma implementação parcial do servidor (server.mjs) e do módulo logHttp (logHttp.mjs).

```
Pedido 1
GET http://localhost:1904/a

Consola do servidor para o pedido 1
Request: GET - /a - {}
Response: 200 - {"a":1,"b":"slb"}

Pedido 2
POST http://localhost:1904/b
Content-Type: application/json
{
 "a":"slb","b": 2
}

Consola do servidor para o pedido 2
Request: POST - /b - {"a":"slb","b":2}
Response: 200 - {"response":"OK"}
```

**server.mjs**

```
1 import express from 'express'
2 import logHttp from './logHttp.mjs'
3 export const app = express()
4
5 app.use(express.json())
6
7 app.use(_____) // 1
8
9 app._____ // 2 ('/a', function (req, rsp) { _____ }) // 3
10 app._____ // 4 ('/b', function (req, rsp) { _____ }) // 5
11
12 app.listen(1904)
```

**logHttp.mjs**

```
1 export default () =>
2 function(req, rsp, next) {
3 console.log(`Request: ${_____ // 1} - ${req.url} - ${JSON.stringify(_____ // 2)}`)
4 const oldJson = _____ // 3
5 rsp.json = function(obj) {
6 console.log(`Response: ${_____ // 4} - ${_____ // 5}`)
7 return _____ // 6
8 }
9 _____ // 7
10 }
```

- a. [2] Complete a implementação do módulo server.mjs.  
b. [2] Complete a implementação do módulo logHttp.mjs.  
c. [1] O que aconteceria se a linha 5 de server.js fosse removida. Justifique no máximo de 2 linhas.

Os docentes,  
Fernanda Passos, Filipe Freitas e Luís Falcão

